

Proyecto Batalla Naval Distribuido

Arquitectura General del Proyecto

Este es un sistema distribuido cliente-servidor que implementa el juego de Batalla Naval usando tecnología RMI (Remote Method Invocation). El proyecto está estructurado como un **multi-módulo Maven** con arquitectura modular.

Estructura de Módulos

1. Módulo **shared** - Componentes Compartidos

```
shared/
├── pom.xml
└── src/main/java/co/edu/uptc/shared/
    ├── interfaces/           # Interfaces RMI
    │   ├── GameCallback.java # Callback para notificaciones
    │   └── GameService.java  # Servicio principal RMI
    └── model/                # Modelos de datos
        ├── Board.java        # Tablero de juego
        ├── GameStats.java    # Estadísticas de partida
        ├── GameStatus.java   # Estado del juego
        ├── PlayerStats.java  # Estadísticas del jugador
        ├── Position.java     # Coordenadas
        └── Ship.java         # Representación de barcos
```

Propósito: Contiene todas las clases e interfaces que deben ser compartidas entre cliente y servidor, incluyendo interfaces RMI y modelos de datos serializables.

2. Módulo **server** - Servidor Distribuido

```
server/
├── pom.xml
└── src/main/java/co/edu/uptc/server/
    ├── GameServiceImpl.java # Implementación del servicio RMI
    ├── GameSession.java     # Gestión de sesiones de juego
    ├── Player.java          # Representación del jugador
    └── ServerMain.java       # Punto de entrada del servidor
```

Propósito: Implementa la lógica del servidor, gestiona las sesiones de juego, coordina múltiples clientes y expone servicios RMI.

3. Módulo **client** - Cliente con GUI

```
client/
├── pom.xml
└── src/main/java/co/edu/uptc/client/
    ├── BoardPanel.java          # Panel del tablero de juego
    ├── ClientMain.java          # Punto de entrada del cliente
    ├── GameCallbackImpl.java    # Implementación de callbacks
    ├── GameController.java      # Controlador de la lógica
    ├── GameStatsPanel.java      # Panel de estadísticas
    ├── GameWindow.java          # Ventana principal
    └── NetworkDiscovery.java     # Descubrimiento de red
```

Propósito: Interfaz gráfica Swing que se conecta al servidor RMI y proporciona la experiencia de usuario.

🔑 Componentes Técnicos Clave

Interfaces RMI (*shared/interfaces/*)

1. **GameService**: Interfaz principal que define los métodos remotos:
 - Conexión de jugadores
 - Colocación de barcos
 - Ataques
 - Gestión de turnos
2. **GameCallback**: Para notificaciones asíncronas del servidor al cliente:
 - Cambios de turno
 - Resultados de ataques
 - Fin de partida

Modelos de Datos (*shared/model/*)

- **GameStats**: Estadísticas detalladas de la partida actual
- **PlayerStats**: Estadísticas históricas del jugador
- **Board**: Representación del tablero 10x10
- **Ship**: Barcos con posiciones y estado
- **Position**: Coordenadas inmutables

Servidor RMI (*server/*)

GameServiceImpl:

- Implementa la interfaz **GameService**
- Gestiona múltiples jugadores concurrentes
- Coordina sesiones de juego
- Sistema de heartbeat para detectar desconexiones

GameSession:

- Representa una partida entre 2 jugadores

- Gestiona turnos y estado del juego
- Valida movimientos

Player:

- Encapsula información del jugador
- Mantiene estadísticas de partida
- Referencia al callback del cliente

Cliente GUI (client/)

GameWindow:

- Ventana principal con dos tableros lado a lado
- Interfaz Swing completa
- Panel de mensajes y controles

GameController:

- Coordina comunicación RMI con el servidor
- Gestiona callbacks y polling de estado
- Maneja la lógica de la aplicación

GameStatsPanel:

- Panel elegante para mostrar estadísticas detalladas
- Comparación entre jugadores
- Tabla con métricas de la partida

Distribución y Despliegue

Compilación

```
mvn clean package # Compila todos los módulos
```

Ejecutables

1. Servidor:

```
java -Djava.security.policy=all.policy -cp shared/target/shared-1.0-SNAPSHOT.jar;server/target/server-1.0-SNAPSHOT.jar  
co.edu.uptc.server.ServerMain
```

2. Cliente:

```
java -Djava.security.policy=all.policy -jar client/target/battleship-client-jar-with-dependencies.jar [host] [port]
```

Paquete Cliente (`paquete_cliente_completo/`)

- **JAR autocontenido:** `battleship-client-jar-with-dependencies.jar`
- **Scripts de inicio:** `INICIAR.bat`, `run-client-lan-simple.bat`
- **Verificador de sistema:** `verificar.bat`
- **Políticas de seguridad:** `all.policy`

Flujo de Juego

1. **Conexión:** Cliente se conecta al servidor RMI
2. **Lobby:** Espera a otro jugador
3. **Preparación:** Cada jugador coloca sus 5 barcos (tamaños: 5,4,3,3,2)
4. **Juego:** Turnos alternados de ataque
5. **Fin:** Cuando todos los barcos de un jugador son hundidos
6. **Estadísticas:** Muestra panel detallado con métricas de la partida

Características Destacadas

- **Sistema distribuido real** usando RMI
- **Callbacks bidireccionales** para notificaciones en tiempo real
- **Polling de respaldo** en caso de fallo de callbacks
- **Estadísticas detalladas** de partida y jugador
- **Interfaz gráfica moderna** con Swing
- **Detección automática de IP LAN**
- **Sistema de heartbeat** para detectar desconexiones
- **Gestión robusta de errores** y timeouts
- **Empaquetado distribuible** listo para LAN

Tecnologías Utilizadas

- **Java RMI** (Remote Method Invocation)
- **Maven** (Gestión de dependencias y construcción)
- **Swing** (Interfaz gráfica)
- **Java Serialization** (Transferencia de objetos)
- **Multi-threading** (Callbacks y heartbeat)

Patrones de Diseño Implementados

- **MVC (Model-View-Controller):** Separación clara entre lógica de negocio y presentación
 - **Observer Pattern:** Callbacks para notificaciones asíncronas
 - **Singleton:** `GameServiceImpl` actúa como punto central
 - **Factory Pattern:** Creación de sesiones de juego
 - **Strategy Pattern:** Diferentes tipos de ataques y validaciones
-